

RELATÓRIO DA AULA 007

Na Aula 007 estudamos expressões, atribuições e estruturas de controle nas linguagens de programação. O objetivo foi entender como os valores são combinados, armazenados e como o programa define a ordem em que os comandos serão executados.

Primeiro, vimos as expressões aritméticas, a precedência e a associatividade dos operadores. A precedência define qual operador será executado primeiro e a associatividade define a ordem quando existem operadores com a mesma prioridade.

Precedência e associatividade na prática

Expressão	Python	JavaScript	Java
<code>-2 ** 2</code>	-4	erro de sintaxe	(não há <code>**</code>)
<code>2 ** 3 ** 2</code>	512	512	—
<code>1 + 2 + "3"</code>	<code>TypeError</code>	"33"	"33"
<code>-7 / 2</code> (inteiros)	-4 (com <code>//</code>)	-3.5	-3
<code>-7 % 2</code>	1	-1	-1

Associatividade (p. 303)
Esquerda: `*` `/` `%` `+` `-`
Direita: `**` (Python, JS, Ruby, Fortran),
`++`, `--`, menos unário
Visual Basic: `^` à esquerda!

Excel
`=- 2^2` dá **4**; lá o menos unário vence o `^`.

Também percebemos que diferentes linguagens podem apresentar resultados diferentes para uma mesma expressão, sendo importante conhecer suas regras e utilizar parênteses quando necessário.

Também estudamos os efeitos colaterais, que acontecem quando uma operação ou função altera algum valor durante sua execução. Vimos ainda a sobrecarga de operadores, em que um mesmo operador pode ter diferentes significados dependendo dos valores utilizados.

Depois, aprendemos sobre conversão de tipos, como alargamento, estreitamento, coerção e cast. As conversões automáticas podem facilitar a programação, mas também podem esconder erros. Em JavaScript e PHP, por exemplo, algumas comparações podem converter os tipos automaticamente e produzir resultados inesperados.

Conversões de tipos: alargamento, estreitamento e coerção

Java · coerção e cast

```
int grande = 123456789;
float f = grande;      // alargamento implícito
System.out.println((int) f); // 123456792 !

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 200); // -56

byte b = 10, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não: float64(i)	erro de compilação
Rust	não: i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em `d = b * a`; com a digitado por engano no lugar de `c`, o Java converte e compila.

Nas expressões booleanas, estudamos valores verdadeiros e falsos, comparações e o funcionamento do curto-circuito. Operadores como `&&` e `||` podem impedir que uma parte da expressão seja executada quando o resultado já pode ser determinado.

Na parte de atribuição, vimos atribuição simples, composta e outras formas de atualizar valores. Também estudamos problemas envolvendo diferentes tipos, como a divisão inteira em Java, em que dois valores `int` são divididos antes da conversão para `double`, podendo resultar em `0.0`.

Por fim, estudamos as estruturas de controle, como `if`, `else`, `switch`, `for` e `while`, além de `break`, `continue` e `do-while`. Vimos problemas como o `dangling else` e o `fall-through` no `switch`, quando a falta de `break` faz a execução continuar nos próximos `case`. Também conhecemos o `goto` e alguns problemas relacionados ao seu uso.

A aula apresentou exercícios para encontrar bugs em códigos e mostrou que erros pequenos podem causar resultados inesperados. Também foram apresentados casos reais, como o “goto fail” da Apple e o problema do Zune causado por um laço infinito. Com isso, entendemos a importância de conhecer as regras da linguagem para escrever códigos mais claros, seguros e confiáveis.

Desvio incondicional: o goto

"goto fail" (Apple, 2014) - versão simplificada

```
int valida_certificado(void) {
    int err;
    if ((err = verifica_hash()) != 0)
        goto fail;
    goto fail;           // duplicada: desvia SEMPRE
    if ((err = verifica_assinatura()) != 0)
        goto fail;       // nunca é alcançada
fail:
    return err;           // 0 = "certificado válido"
}
```

Dijkstra (1968)

"Go To Statement Considered Harmful"

Quem tem goto hoje

C, C++, C#, Go. Java **não** tem (p. 353)

Gotos restritos

break, continue e return melhoram a legibilidade (p. 353)

Com chaves obrigatórias (Go, Rust) ou com gcc -Wall (aviso de endentação enganosa), o bug teria sido pego.

EXERCÍCIOS DE BUGS

Exercício em duplas: encontre o bug (1/3)

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

1 - JavaScript

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
    total += p;
console.log("Total: " + total);
```

2 - Python

```
def desconto(preco, d=None):
    d = d or 10    # padrão: 10%
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

Exercício: encontre o bug (2/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

3 · Java

```
int dia = 2; String nome = "";
switch (dia) {
    case 1: nome = "domingo";
    case 2: nome = "segunda";
    case 3: nome = "terça";
    default: nome = "inválido";
}
System.out.println(nome);
```

4 · C

```
int saldo = 100, saque = 50;
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

Exercício: encontre o bug (3/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

5 · Go

```
x := 10
if x > 5 {
    x := x * 2
    fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

6 · Java

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

- 1. JavaScript** – (a) A saída é "0012". (b) O problema acontece porque o `for...in` percorre os índices do array, e não os valores. (c) Corrigir utilizando `for...of`.
- 2. Python** – (a) A saída é 10. (b) O problema acontece porque o valor 0 é considerado falso em Python. (c) Corrigir verificando se `d` é `None`.
- 3. Java** – (a) A saída contém os resultados dos case seguintes ao escolhido. (b) O problema acontece porque não há `break`, causando *fall-through*. (c) Corrigir adicionando `break` ao final de cada case.

4. C –(a) O comando seguinte ao if é executado independentemente da condição. (b) O problema acontece devido ao ; colocado após o if. (c) Corrigir removendo o ;.

5. Go – (a) A variável externa não é alterada, pois uma nova variável é criada dentro do escopo. (b) O problema acontece por causa do *shadowing* causado pelo uso de :=. (c) Corrigir utilizando = para alterar a variável existente.

6. Java – (a) A saída é 0.0. (b) O problema acontece porque a divisão entre dois int é realizada como divisão inteira antes da conversão para double. (c) Corrigir fazendo a conversão antes da divisão, como em (double) a / b.